

Blue-Green Deployment Implementation

Automated Weighted Traffic Routing using Application Load Balancers and Auto Scaling Groups

1. Executive Summary & Architecture

This document outlines the deployment strategy for an AWS infrastructure using Terraform to implement a zero-downtime **Blue-Green Deployment**. By leveraging an Application Load Balancer (ALB) with weighted target groups, incoming traffic is seamlessly transitioned between two identical isolated environments: Blue (Active) and Green (Staging/Next Release).

Blue Environment (Current)

- **Target Group:** cmtr-8k07hv2y-blue-tg
- **Auto Scaling Group:** cmtr-8k07hv2y-blue-asg
- **Launch Template:** cmtr-8k07hv2y-blue-template
- **Initial Weight:** 100%
- **Response Content:** Blue Environment

Green Environment (Target)

- **Target Group:** cmtr-8k07hv2y-green-tg
- **Auto Scaling Group:** cmtr-8k07hv2y-green-asg
- **Launch Template:** cmtr-8k07hv2y-green-template
- **Initial Weight:** 0%
- **Response Content:** Green Environment

2. Key Technical Fixes Applied

During initial deployment attempts, resource conflicts and race conditions occurred. The following key adjustments resolved all deployment errors:

A. Pre-Existing Infrastructure Clean-up

When Terraform attempts to create resources that were left behind from previous un-tracked runs (e.g., target groups or load balancers with matching names), AWS rejects creation. Cleaning up the stale resources manually via AWS CLI or Console allowed Terraform to cleanly manage the complete lifecycle.

B. Explicit Dependency Ordering (depends_on)

Auto Scaling Groups depend on the ALB HTTP Listener to exist and be active prior to registering target group associations. Adding explicit dependencies ensures smooth provisioning order:

```
resource "aws_autoscaling_group" "blue" { ... depends_on = [aws_lb_listener.http] }
```

C. Exact Launch Template Naming

Replacing dynamic `name_prefix` attributes with deterministic name declarations prevents orphaned dynamic templates from lingering in AWS and causing conflicts.

3. Step-by-Step Execution Workflow

Step 1: Code Formatting and Validation

Always ensure your code meets standard formatting rules and contains valid dependency trees before planning.

```
terraform fmt terraform validate
```

Step 2: Dry-Run Plan & Initial Deployment

Deploy the infrastructure with initial weights (Blue = 100, Green = 0):

```
terraform plan terraform apply -auto-approve
```

Step 3: Traffic Routing Shift (Green Switchover)

To shift traffic to the Green environment, update `terraform.tfvars`:

```
blue_weight = 0 green_weight = 100
```

Apply the change to update the ALB Listener weights without touching EC2 instances:

```
terraform apply -auto-approve
```

Critical Verification Step for Platform Submission

The evaluation framework verifies that your repository contains the final valid Terraform code, but requires all created AWS resources to be destroyed prior to clicking **Verification**.

Step 4: Clean Up Infrastructure

Destroy all created AWS infrastructure before submitting verification:

```
terraform destroy -auto-approve
```

4. Terraform File Configuration Summary

- `versions.tf`: Requires Terraform `>= 1.5.7` and configures the AWS provider.
- `variables.tf`: Declares typed input variables (e.g., `blue_weight`, `green_weight`) with full descriptions.